

Разработка на библиотека за обектно-реляционно картографиране по време на компилация

Дипломна работа

Алекс Цветанов, фак. № 121222225

Научен ръководител: проф. д-р инж. Румен Трифонов

Технически университет -- София
Факултет „Компютърни системи и технологии“
Компютърно и софтуерно инженерство

2026

Проблемът със заявките, проверявани при изпълнение

Традиционен подход:

- Заявките се описват като **низове**
- Валидиране едва при изпълнение
- Грешни имена на полета → сив при изпълнение
- Несъвместими типове → късно откриване
- Смяна на хранилището → **пълно пренаписване**

Изследователски въпрос:

Може ли компилаторът да поеме ролята на първи валидатор на

При изпълнение	При компилация
Низови заявки	<code>constexpr</code> междинно представяне
Грешка при изпълнение	Грешка при компиляция
Късно откриване	Ранно откриване

Цел

Компилаторът поема ролята на първи валидатор на заявките.

Съществуващи решения и позиция на разработката

Решение	Подход	Ограничение
SOI ODB	при изпълнение кодогенерация	низови заявки външен генератор
sqlpp11 Hiberlite	вграден DSL ORM	главно за SQL само за SQLite

Цел на разработката

- `constexpr` междинно представяне
- Договор на конектора и способности
- 7 системи + корутинен `async` слой

Основна теза

Един C++ модел, валидиран при компилация, разширяем към много системи и практически приложим при асинхронни натоварвания.

Защо тази комбинация е уникална

Отличителна ос	Какво липсва другаде
<code>constexpr</code> IR, валидиран при компилация	sqlpp11: само SQL
Без външна кодогенерация	ODB: външен генератор
Един модел: рел. + нерел. (7 системи)	Hiberlite: само SQLite
Договор за способности (gating при компилация)	никой
Корутинна <code>async</code> ос	никой

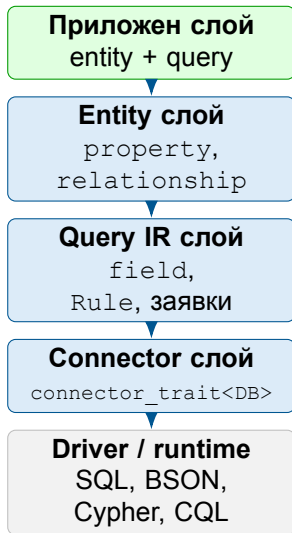
Приносът

Всяка ос съществува някъде *поотделно*. Комбинацията от петте не съществува в нито една C++ библиотека за достъп до данни.

Видяно на живо (Демо 1)

Една заявка с `.join()`: релационен конектор я приема, документен я *отказва при компилация*.

Слоеста архитектура на библиотеката



Ключови архитектурни принципи:

- Без виртуален интерфейс --- признаков (trait-based) договор
- `constexpr` междинно представяне на заявката --- ранна валидация
- Маркери за способности (capability tags) ограничават неподдържаните операции
- Смяна на системата за съхранение = смяна само на типа

Централен инвариант

Приложният код не комуникира директно с драйвера --- работи само с C++ типове.

Минимален compile-time пример

```
struct User {  
    orm::property<int, "id"> id;  
    orm::property<std::string, "name">  
        name;  
};  
  
static constexpr auto q =  
    orm::select(  
        orm::field<&User::id>,  
        orm::field<&User::name>  
    ).where(orm::field<&User::id>  
        == orm::Placeholder<int>{{}});
```

- `property<T, "name">` носи тип и compile-time име на колона
- Заявката се изгражда като `constexpr IR`, а не като низ
- Грешно поле или неподдържана операция се открива рано

Ключова идея

Компиляторът поема ролята на първи валидатор на заявката.

Договор на конектора и механизъм на способностите

Минимален договор на конектора:

```
template<>
struct orm::connector_trait<MyDB> {
    template<typename T> struct
        wire_type;
    using cursor_type = ...;

    using supports_transactions =
        true_type;

    static auto execute(
        MyDB& db, auto query,
        auto&&... params);
};
```

- `is_connector<DB>` валидира интерфейса при компилация
- Маркерите за способности ограничават неподдържаните операции
- Смяната на системата за съхранение е смяна на типа, а не на интерфейса за заявки

Демонстрирани системи за съхранение

SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j, Cassandra

Един модел, много системи за съхранение

Едно междинно представяне → различни системи:

Система	Материализация
SQLite	WHERE id = ?
PostgreSQL	WHERE id = \$1
MySQL	WHERE id = ?
MongoDB	BSON filter
Redis	GET/SET/DEL
Neo4j	Cypher MATCH...WHERE
Cassandra	CQL SELECT...FROM

Абстракция на връзките:

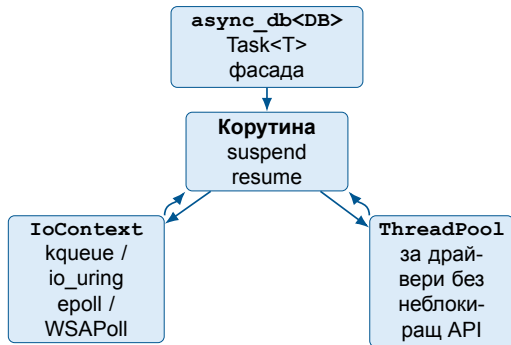
reference	SQL:	JOIN
	MongoDB:	\$lookup
embed	MongoDB	вложен документ

Избор на поле от свързана таблица → JOIN-ът се **извежда при компилация** и връща `joined_row`; еднакво за SQL и NoSQL --- **живо в Демо 7.**

Принцип

Един C++ модел е *логическата* схема; хранилището определя материализацията. Едно приложение
macOS / Linux / Windows

Асинхронна архитектура на изпълнението



- Работната нишка не блокира; корутината се преустановява и продължава при готовност
- **Неблокиращ път:** PostgreSQL, MySQL/MariaDB, Cassandra, Redis
- **Обединителен път:** SQLite, MongoDB, Neo4j

Ключов извод

Асинхронният слой е втора архитектурна ос, а не обвивка за удобство.

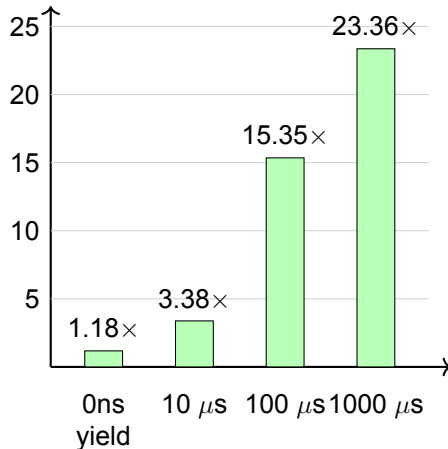
Верификационна стратегия

Доказателствена база

271 теста --- 100% преминаване
при компилация + единични +
интеграционни + срещу живи системи

- Междинно представяне, способности, миграции, нишкова безопасност
- Асинхронна инфраструктура: `Task<T>`, `ThreadPool`, `IoContext`
- Емпирични измервания в `bench_async.cpp`

ускорение async/sync



4 нишки, 100 задачи --- ефектът расте с изчакването

Основни резултати и извод

- 1 ORM модел с проверка при компилация в стандартен C++23, без външна кодогенерация
- 2 Един C++ обектен модел за 7 системи за съхранение (SQL и NoSQL)
- 3 **Релационно-осъзнати заявки:** авто-JOIN, изведен при компилация; една заявка → SQL JOIN / Mongo \$lookup, жив `joined_row` от 4 бази
- 4 Формален договор на конектора и разширяемост чрез способности
- 5 Асинхронен слой с измерима полза при натоварвания, ограничени от вход-изход
- 6 Многостепенна верификация: тестове + проверки срещу живи системи + емпирични измервания

Основен извод

Трета алтернатива освен ORM при изпълнение (късно откриване) и пряк достъп през драйвера (без абстракция): договор на конектора, валидиран от компилатора, плюс корутинно изпълнение. **Тази комбинация не съществува в друга C++ библиотека.**

Един и същ код --- **macOS / Linux / Windows**, релационни и документни бази; демонстрациите вървят „out of the box“ през Docker. **Бъдещо**

Демонстрации — компилаторът като първи валидатор

Седем демонстрации: шест с предварително *уловен* изход (без жива компилация на сцената)
+ едно *живо* multi-store изпълнение през Docker.

Демонстрира

- | | |
|--------|--|
| Демо 1 | MySQL приема, MongoDB отказва JOIN |
| Демо 2 | Заявката е <code>constexpr</code> стойност, не низ |
| Демо 3 | Несъвместим тип поле ↔ стойност |
| Демо 4 | Грешен брой/тип на параметрите |
| Демо 5 | Redis: само по първичен ключ |
| Демо 6 | relationship движи заявката (авто-JOIN, MockDB) |
| Демо 7 | Една заявка, четири живи бази (SQL × NoSQL) |
-

Обща нишка: грешки, които другаде се проявяват при изпълнение, тук са отказ на компилатора.

Демо 1 — Един модел: MySQL приема, MongoDB отказва

Една и съща SELECT ... JOIN заявка срещу два конектора:

```
auto q = select(field<&User::id>, field<&Post::body>)
    .join<inner, Post>(field<&User::id> == field<&Post::author_id>);

db<MySQLDB>{conn} << q;    // relational -> compiles
db<MongoDB>{conn} << q;    // document    -> REJECTED
```

```
$ c++ -std=c++23 -I lib/include -DDEMO_REJECT capability_gating.cpp
```

```
lib/include/ORM/connector/db.hpp:35:35: error: static assertion failed due to
  requirement 'has_capability<orm::MongoDB, orm::cap::supports_joins>': orm::db: this
  connector does not support JOIN operations. Remove .join() clauses, use
  store_as::reference relationships, or switch to a connector that declares `using
  supports_joins = void;`.
1 error generated.
```

Демо 2 — Заявката е constexpr стойност, не низ

Структурата е в типа и се проверява при компилация --- нищо не се изпълнява.

```
constexpr auto q = select(field<&User::id>, field<&User::name>)  
    .where(field<&User::score> > 0.0);  
  
static_assert(decltype(q)::response_type::size == 2);  
static_assert(decltype(q).where_clauses())::size == 1);
```

Имена на колони, проекция и клаузи --- проверени без изпълнение. (SOI гради низ при изпълнение.)

Демо 3 — Несъвместим тип поле↔стойност

Операторите сверяват C++ типа на колоната и на стойността:

```
field<&User::score> > 0; // double vs int -> OK  
field<&User::id> == std::u8string{u8"x"}; // int vs string ->  
REJECTED
```

```
$ c++ -std=c++23 -I lib/include -DDEMO_REJECT type_safety.cpp
```

```
lib/include/ORM/query/rules.hpp:121:5: error: static assertion failed due to  
  requirement 'detail::comparable_value_types<int, std::u8string>': orm: incompatible  
  operand types in a field comparison. The right-hand value's C++ type is not  
  convertible to (or from) the column's C++ type. Check the literal/placeholder type  
  against the property<> declaration of the field.  
1 error generated.
```

Демо 4 — Брой и тип на параметрите

Заявка с два анонимни placeholder-а, извикана с грешен брой аргументи:

```
auto q = select(field<&User::id>
    .where( (field<&User::id>      == Placeholder<int>{}) &&
            (field<&User::score> > Placeholder<double>{}) ) );

db.execute(q, 7, 3.5);    // 2 args -> OK
db.execute(q, 7);         // 1 arg  -> REJECTED
```

```
$ c++ -std=c++23 -I lib/include -DDEMO_REJECT param_arity.cpp
```

```
lib/include/ORM/connector/param_check.hpp:202:21: error: static assertion failed due to
requirement 'n_ph == sizeof...(Params)': orm: wrong number of runtime parameters
passed to execute(). The argument count does not match the number of anonymous
placeholders (orm::Placeholder<T>) in the query's clauses. Note: queries with no
placeholders should be executed via `db << query`, and indexed placeholders
(orm::ph<T, _N>) are validated by the connector.
lib/include/ORM/connector/param_check.hpp:202:26: note: expression evaluates to '2 == 1'
1 error generated.
```


Демо 5 — Redis: само по първичен ключ

Заявка с не-ПК предикат срещу Redis конектора:

```
select (field<&Session::token>)
  .where (field<&Session::id> == Placeholder<int>{});           // PK ->
OK
```

```
select (field<&Session::token>)
  .where (field<&Session::id> == Placeholder<int>{})
  .where (field<&Session::token> == Placeholder<u8string>{}); //
non-PK -> REJECTED
```

```
$ c++ -std=c++23 -I lib/include -DDEMO_REJECT backend_shape.cpp
```

```
lib/include/ORM/db/connectors/RedisDB/redis_db.hpp:107:27: error: static assertion
  failed due to requirement 'redis_detail::is_pk_only_where()': RedisDB connector
  supports only primary-key equality WHERE predicates
1 error generated.
```

Демо 6 — relationship движи заявката (автоматичен JOIN)

`relationship<reference<&User::id>, "user_id">` е истинска FK колона; избор на поле от свързана таблица извежда JOIN-а при компилация:

```
struct Post {  
    property<int, "id"> id;    property<u8string, "body"> body;  
    relationship<store_as::reference<&User::id>, "user_id"> user_id;  
};  
  
db << select(field<&Post::id>, field<&User::name>);
```

→ изведен и проверен при компилация SQL:

```
SELECT posts.id, user.name  
FROM posts INNER JOIN user ON posts.user_id = user.id
```

`optional_field` → LEFT / RIGHT / FULL (матрица 2×2); многозвенно `Comment` → `Post` → `User` се извежда автоматично.

Демо 7 — Една заявка, четири живи бази (SQL × NoSQL)

Демо 6 показва *рендерирането*; тук същата заявка се *изпълнява* срещу живи бази през `docker compose` --- релационни *и* документна:

```
constexpr auto q =
    select(field<&Book::title>, field<&Author::name>);
auto rows = (db << q).to_vector(); // vector<joined_row<Book, Author>>
```

→ изпълнено срещу всяко хранилище (един и същ C++):

```
SQLite / PostgreSQL / MySQL : INNER JOIN authors ON books.author_id = authors.id
MongoDB                       : $lookup { from: authors, localField: author_id,
                                         foreignField: id } + $unwind
all four -> joined_row{ "The Hobbit"->Tolkien,
                        "A Wizard of Earthsea"->Le Guin }
```

Един модел, една заявка → SQL JOIN *и* \$lookup, идентичен joined_row. „Out of the box“ през Docker --- еднакво на macOS / Linux / Windows.

Благодаря за вниманието!

Въпроси?

Какво още може да се провери при компилация

Вече се проверява:

- Договор и способности на конектора
- Тип на заявката и видове клаузи
- Обвързване поле ↔ таблица ↔ колона
- Консистентност на проекцията
- Валидност на placeholder-ите
- Структурна форма за конкретния backend
- **Съвместимост поле ↔ стойност по тип**
- **Брой/тип на параметрите (анонимни)**

Две скорошни проверки --- всяка е отказ при компилация:

```
field<&User::id> == std::u8string{u8"x"};    // incompatible type
db.execute(query_with_2_placeholders, 5);    // wrong arg count
```

Архитектурно възможно (надграждане):

- Аритет при *индексирани* параметри
--- по дефиниция договор на конектора
- Compile-time валидиране на enum_t/set_t
- Референтна цялост на заявката
- Деривирание на схемата чрез C++26 reflection

Грешки, уловени при компилация

Клас грешка	Уловена при компилация чрез
Неподдържана операция за backend (JOIN срещу MongoDB)	<code>static_assert</code> за липсваща <code>supports_joins</code>
Несъществуващо / грешно име на колона	<code>поле = член-указател &T::m; име от <code>property<T, "..."></code></code>
Достъп до неселектирана колона	<code>static_assert: „field not found in projection“</code>
Малформирана заявка	<code>concepts is_field/is_rule</code>
Несъвместим тип на сравнение (<code>int</code> колона vs <code>string</code>)	<code>static_assert: „incompatible operand types“</code>
Невалиден индекс на placeholder	<code>requires std::is_placeholder<...></code>
Нарушена форма за backend (Redis / Cassandra)	<code>constexpr + static_assert</code>
Неспециализиран конектор	<code>static_assert B connector_trait<DB></code>

Всяка от тези грешки в традиционните ORM библиотеки би била изключение или SQL грешка при изпълнение; тук е отказ на компилатора.

Матрица на диференциация (уникалност)

Ос		SOCI	sqlpp11	ODB	Hiberlite	Diesel	ТАЗИ
constexpr IR при компиляция		не	част.	част.	не	част.	да
Без външна кодогенерация		да	да	не	да	не	да
Един модел: рел. + нерел.		не	не	не	не	не	да
Договор за способности (gating)		не	не	не	не	не	да
Корутинна async ос		не	не	не	не	част.	да

Всеки конкурент има поне една „да“ клетка; никой няма пълния ред --- това е приносът. (Diesel е Rust; Hibernate/SQLAlchemy --- в ръководството.)